



Exercices — Jour 2

React Native · Modules 3 & 4 : Architecture + Interface

Progression : les exercices 1-3 portent sur l'architecture (state, Redux, Zustand), les exercices 4-6 sur l'interface (navigation, Flexbox, animations). L'exercice 7 est un projet intégrateur.

● Exercice 1 — useReducer : Panier d'achat (25 min)

Objectif : gérer un state complexe avec `useReducer` sans bibliothèque externe.

Consigne

Créez `app/panier.tsx` — un panier d'achat simple.

Actions à implémenter :

```
type Action =
  | { type: 'AJOUTER';    payload: Produit }
  | { type: 'RETIRER';    payload: string } // id du produit
  | { type: 'QUANTITE';   payload: { id: string; quantite: number } }
  | { type: 'VIDER' };

interface Produit {
  id: string;
  nom: string;
  prix: number;
  quantite: number;
}
```

Fonctionnalités :

1. Afficher la liste des produits avec quantité et sous-total
2. Boutons `+` / `-` pour ajuster les quantités (min 1)
3. Bouton `🗑` pour retirer un article
4. Afficher le **total** en bas

5. Bouton "Vider le panier"

Reducer à compléter :

```
function panierReducer(state: Produit[], action: Action): Produit[] {  
  switch (action.type) {  
    case 'AJOUTER':  
      // Si déjà dans le panier, incrémenter la quantité  
      // Sinon ajouter avec quantite = 1  
      return []; // À compléter  
    case 'RETIRER':  
      return []; // À compléter  
    case 'QUANTITE':  
      return []; // À compléter  
    case 'VIDER':  
      return []; // À compléter  
    default:  
      return state;  
  }  
}
```

✓ Critères de validation

- Aucune mutation directe du state (pas de `push`, `splice` ...)
- Le total se recalcule automatiquement (`useMemo`)
- La quantité ne peut pas descendre en dessous de 1

● Exercice 2 — Redux Toolkit : Gestion de tâches (30 min)

Objectif : configurer un store Redux complet et le connecter à l'interface.

Consigne

Créez une application de gestion de tâches avec Redux Toolkit.

Structure du store :

```
// store/tasksSlice.ts
interface Tache {
  id: string;
  titre: string;
  description: string;
  priorite: 'basse' | 'normale' | 'haute';
  faite: boolean;
  creeLe: number; // timestamp
}
```

Slice à créer avec les reducers :

- ajouter(state, action: PayloadAction<Omit<Tache, 'id' | 'faite' | 'creeLe'>>)
- basculerFaite(state, action: PayloadAction<string>)
- modifierPriorite(state, action: PayloadAction<{ id: string; priorite: Tache['priorite'] }>)
- supprimer(state, action: PayloadAction<string>)
- toutFaire(state)

Sélecteurs à créer :

```
// store/tasksSlice.ts – sélecteurs
export const selectTachesFaites = (state: RootState) => ...;
export const selectTachesEnCours = (state: RootState) => ...;
export const selectTachesParPriorite = (priorite: string) => ...;
```

Interface à développer :

1. Formulaire d'ajout (titre, description, priorité)
2. Liste des tâches avec indicateur de priorité coloré
3. Checkbox pour marquer comme faite
4. Filtre par statut (toutes / en cours / terminées)
5. Compteur "X tâches restantes"

Critères de validation

- Le store est typé avec RootState et AppDispatch
- Les hooks useAppSelector et useAppDispatch sont utilisés (pas les versions génériques)
- Immer est utilisé correctement dans les reducers (mutations directes OK dans le slice)

Exercice 3 — Zustand + Persistance (25 min)

Objectif : créer un store Zustand avec persistance AsyncStorage.

Consigne

Refactorisez l'exercice 2 (ou créez un nouveau store) avec Zustand.

```
// store/usePreferencesStore.ts
interface Preferences {
  theme: 'light' | 'dark' | 'auto';
  langue: 'fr' | 'en' | 'es';
  notificationsActives: boolean;
  taillePolice: 'petite' | 'normale' | 'grande';
  // Actions
  setTheme: (theme: Preferences['theme']) => void;
  setLangue: (langue: Preferences['langue']) => void;
  toggleNotifications: () => void;
  setTaillePolice: (taille: Preferences['taillePolice']) => void;
  resetPreferences: () => void;
}
```

Fonctionnalités :

1. Store persistant avec `AsyncStorage` via middleware `persist`
2. Écran de préférences avec tous les réglages
3. Affichage dynamique selon les préférences (thème, taille police)
4. Bouton "Réinitialiser" qui revient aux valeurs par défaut

Tester la persistance :

- Modifier une préférence
- Fermer et rouvrir l'app
- Vérifier que le réglage est conservé

Bonus

- Ajouter TanStack Query pour charger des données depuis <https://jsonplaceholder.typicode.com/users> et les afficher selon la langue choisie

✓ Critères de validation

- Les préférences survivent à un rechargement de l'app
- L'interface reflète immédiatement les changements
- `resetPreferences` restaure exactement les valeurs par défaut

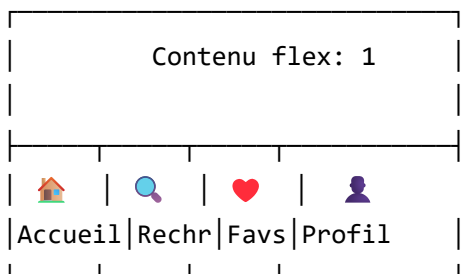
● Exercice 4 — Flexbox : Reproduire des layouts (30 min)

Objectif : maîtriser Flexbox en reproduisant des mises en page réelles.

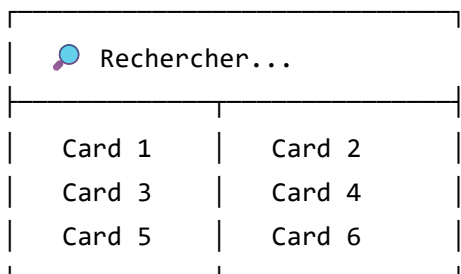
Consigne

Reproduisez les 4 layouts suivants **sans utiliser de positions absolues** :

Layout A — Barre de navigation bas de page



Layout B — Grille de cards 2 colonnes avec header



Layout C — Profil centré avec avatar et stats

[Avatar]			
Prénom NOM			
12 posts	45k	23	suivi
[Suivre]	[Message]		

Layout D — Chat (messages alignés)

[Alice] Bonjour !		
	Salut ! Comment ?	[Moi]
[Alice] Ça va bien, merci		

✓ Critères de validation

- Aucune position: 'absolute'
- Les layouts sont responsive (tester en landscape)
- Le Layout D distingue visuellement les messages envoyés/reçus

● Exercice 5 — Navigation Expo Router (30 min)

Objectif : mettre en place une navigation complète avec Expo Router.

Consigne

Créez une application avec la structure de navigation suivante :

```

app/
├── _layout.tsx           → Tabs : Accueil | Recherche | Profil
├── (tabs)/
│   ├── index.tsx        → Accueil : liste d'articles
│   ├── recherche.tsx    → Recherche avec TextInput
│   └── profil.tsx       → Profil utilisateur
├── article/
│   └── [id].tsx          → Détail article (route dynamique)
└── (auth)/
    ├── connexion.tsx    → Formulaire de connexion
    └── inscription.tsx  → Formulaire d'inscription

```

Fonctionnalités à implémenter :

1. **Tabs** avec icônes Ionicons, couleur active #0d47a1
2. **Liste d'articles** depuis <https://jsonplaceholder.typicode.com/posts>
3. **Tap** sur article → navigation vers `/article/[id]` avec passage du titre en paramètre
4. **Recherche** filtre les articles par titre en temps réel (`useMemo`)
5. **Profil** : bouton "Déconnexion" qui redirige vers `/(auth)/connexion`

Route dynamique :

```

// app/article/[id].tsx
import { useLocalSearchParams } from 'expo-router';

export default function ArticleDetail() {
  const { id, titre } = useLocalSearchParams<{ id: string; titre: string }>();
  // Charger l'article complet : https://jsonplaceholder.typicode.com/posts/${id}
}

```

Bonus

- Ajouter un **Drawer** pour les paramètres
- Protéger les routes tabs avec une vérification d'auth (Zustand)

✓ Critères de validation

- La navigation arrière fonctionne partout
- Les paramètres sont typés avec les génériques de `useLocalSearchParams`
- Le deep link fonctionne (tester `expo-router://article/5`)

● Exercice 6 — Animations : Cards interactives (35 min)

Objectif : implémenter des animations fluides avec Reanimated 3.

Consigne

Créez une interface de cards interactives type "Tinder" :

```
npm install react-native-reanimated react-native-gesture-handler
```

Fonctionnalités :

1. **Stack de cards** empilées (3 visibles, légèrement décalées)
2. **Swipe horizontal** : droite =  accepté, gauche =  refusé
3. **Rotation** : la card pivote selon la direction du swipe
4. **Indicateurs** : icône  ou  apparaît pendant le swipe
5. **Snap** : si swipe > 40% de l'écran → la card part, sinon elle revient

```
// Structure de base
import Animated, { useSharedValue, useAnimatedStyle,
  withSpring, runOnJS } from 'react-native-reanimated';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

function SwipeCard({ item, onSwipeLeft, onSwipeRight }) {
  const translateX = useSharedValue(0);
  const translateY = useSharedValue(0);

  const panGesture = Gesture.Pan()
    .onUpdate((e) => {
      translateX.value = e.translationX;
      translateY.value = e.translationY / 4; // Amortissement vertical
    })
    .onEnd((e) => {
      const SEUIL = 150;
      if (e.translationX > SEUIL) {
        // Swipe droite → accepter
        translateX.value = withSpring(500, {}, () => runOnJS(onSwipeRight)());
      } else if (e.translationX < -SEUIL) {
        // Swipe gauche → refuser
        translateX.value = withSpring(-500, {}, () => runOnJS(onSwipeLeft)());
      } else {
        // Revenir au centre
        translateX.value = withSpring(0);
        translateY.value = withSpring(0);
      }
    });
  // À compléter : useAnimatedStyle pour transform (translateX, rotate)
}
```

Bonus

- `LayoutAnimation` pour la card suivante qui monte
- Compteur d'acceptés / refusés
- Animation de rebond sur les boutons

✓ Critères de validation

- Les animations tournent sur le **UI thread** (pas de janks à 60fps)
- Le `runOnJS` est utilisé pour mettre à jour le state React depuis le worklet

- La rotation est proportionnelle à la translation (pas un palier fixe)

● Exercice 7 — Projet intégrateur : Application complète (60 min)

Objectif : assembler tous les concepts du Jour 2 dans une application cohérente.

Consigne — Application "HackerNews Reader"

Construisez un lecteur d'articles tech en utilisant l'API HackerNews.

API gratuite, sans clé :

Top stories : <https://hacker-news.firebaseio.com/v0/topstories.json>

Article : <https://hacker-news.firebaseio.com/v0/item/{id}.json>

Architecture requise :

```
store/  
├─ articlesSlice.ts      → Redux : articles mis en cache  
└─ favorisSlice.ts      → Redux : articles sauvegardés  
  
app/  
├─ (tabs)/  
│   ├─ index.tsx        → Top stories (FlatList paginée)  
│   ├─ favoris.tsx      → Articles sauvegardés (local)  
│   └─ parametres.tsx    → Thème, préférences (Zustand)  
└─ article/[id].tsx     → Détail + WebView pour l'URL
```

Fonctionnalités minimum :

1. Liste des 30 premiers top stories avec titre, score, auteur, date relative
2. Pagination infinie : charger 10 articles supplémentaires à chaque fin de liste
3. Tap → ouvrir l'URL dans une WebView ou `Linking.openURL`
4. Bouton ★ pour sauvegarder / désauvegarder (persisté avec MMKV ou AsyncStorage)
5. Onglet favoris avec les articles sauvegardés
6. Mode sombre fonctionnel via Zustand + `useColorScheme`

Bonus :

- Animations d'entrée des articles (Moti ou Reanimated)
- Pull-to-refresh
- Recherche dans les favoris



Critères de validation

- Architecture Redux + Zustand cohérente (Redux pour données serveur, Zustand pour préférences)
- Pas de re-renders inutiles (`React.memo` , `useCallback`)
- Gestion des états `loading` , `error` , `empty`
- L'application fonctionne en mode offline pour les favoris